

INFORME DE REQUISITOS

Jean Steven Castañeda Sánchez

Yeferson Estiven Castañeda

Miguel Samudio

Luis Loaiza García

Andrés Vélez Hurtado

Célula 5

CESDE

Técnico Laboral en Desarrollo de Software

Jaime Zapata

Medellín, Colombia

2026

Tabla de contenido

Tabla de contenido	2
Lista de tablas	4
Lista de ilustraciones	5
Glosario	6
Objetivos	7
Objetivo general	7
Objetivos específicos	7
Introducción	8
Planteamiento del problema	10
Necesidad	10
Manejo actual	10
Solución propuesta: Planificador de Menús y Lista de Mercado	10
Requisitos del sistema	11
Requisitos funcionales	11
Requisitos no funcionales	12
Arquitectura y relación Frontend–Backend	13
Despliegue	13
Modelo de base de datos	15
Regla de cálculo de la lista	15
Diagramación	16
Casos de uso	16
Modelo relacional	16
Alcance	18
Entregable final	18
Inclusiones	18
Exclusiones	18
Stack tecnológico	20
Implementación y condiciones legales	21
Naturaleza académica y delimitación de marca	21
Propiedad industrial y uso de signos distintivos	21
Posibles consecuencias jurídicas	22
Derechos de autor sobre el software	23
Responsabilidad de aprendices e institución	23
Cláusula de delimitación propuesta	24
Organización del equipo y metodología	25
Conclusiones como equipo de desarrollo	26
Reflexión final sobre el trabajo en equipo	27
Referencias:	29
Anexo A. Matriz resumida de alcance	32

Lista de tablas

Lista de ilustraciones

Actualizar campo en Word

Glosario

API. Interfaz que permite la comunicación entre componentes de software.

Backend. Capa del sistema responsable de reglas de negocio, procesamiento y acceso a datos.

Frontend. Capa de presentación con la que interactúa directamente la persona usuaria.

CRUD. Operaciones básicas de creación, consulta, actualización y eliminación de datos.

Despensa. Conjunto de ingredientes disponibles antes de generar una lista de mercado.

Lista de mercado. Relación consolidada de ingredientes y cantidades que deben adquirirse.

Plan de menú. Estructura semanal que organiza recetas por día y por tipo de comida.

Receta. Conjunto de ingredientes, cantidades e instrucciones para preparar un plato.

Sprint. Periodo de trabajo acotado utilizado para planificar y entregar incrementos del producto.

Scrum Master. Rol que facilita el marco Scrum, remueve impedimentos y ayuda a que el equipo mantenga el proceso.

SQL. Lenguaje utilizado para consultar y gestionar bases de datos relacionales.

REST. Estilo arquitectónico utilizado para exponer recursos mediante servicios web.

Objetivos

Objetivo general

Definir, documentar y estructurar los requisitos funcionales, no funcionales, técnicos, de alcance y legales para el desarrollo de un Planificador de Menús y Lista de Mercado que permita organizar recetas durante una semana, consolidar ingredientes, descontar existencias disponibles en la despensa y generar una lista de mercado lista para su uso.

Objetivos específicos

- Identificar la necesidad que origina el producto y describir la situación actual que se busca mejorar.
- Definir los actores, casos de uso y reglas de negocio principales del sistema.
- Establecer una arquitectura básica de frontend, backend y base de datos que sea viable para un proyecto académico.
- Modelar cinco tablas relacionales capaces de sostener la planificación semanal, la relación entre recetas e ingredientes y el cálculo de compras.
- Documentar el alcance, las inclusiones, las exclusiones y el entregable final correspondiente a la Célula 5.
- Establecer condiciones legales y de propiedad intelectual que eviten la asociación no autorizada del proyecto con marcas, identidad corporativa, activos o información reservada de terceros.

Introducción

El presente Informe de Requisitos establece la base funcional, técnica y documental para el desarrollo de un sistema denominado Planificador de Menús y Lista de Mercado. La propuesta parte de una necesidad cotidiana: pasar de una receta aislada a una planificación semanal que permita decidir qué preparar, identificar los ingredientes requeridos y determinar qué hace falta comprar.

La solución se plantea como una aplicación web con un frontend desarrollado en React y un backend desarrollado en Spring Boot, apoyados por una base de datos relacional. El frontend ofrecerá un calendario semanal con interacción de arrastrar y soltar para buscar recetas por día y comida, además de una lista de mercado agrupada por categorías con posibilidad de marcar elementos como comprados. El backend será responsable de la creación de planes, consolidación de ingredientes, cálculo de cantidades faltantes y generación de la lista.

Dentro del marco del desarrollo de software bajo metodologías ágiles, la planificación técnica y la estructuración de los datos constituyen un pilar fundamental para garantizar la escalabilidad, mantenibilidad y correcta integración entre los componentes del sistema. El presente documento detalla el diseño del modelo de base de datos relacional correspondiente al Lote 05, enfocado en el "Planificador de Menús y Lista de Mercado" (Célula 5). Este módulo permite evolucionar desde la selección individual de recetas hasta la estructuración de un plan semanal automatizado, calculando faltantes de despensa y generando de forma inteligente la lista de compras.

El documento también establece el alcance del entregable de la Célula 5, la propuesta de modelo de datos, los diagramas de casos de uso y modelo relacional, el stack tecnológico, los criterios de despliegue y las condiciones legales aplicables. En particular, se establece una

separación explícita entre un proyecto académico y cualquier empresa, marca o identidad corporativa que pueda haber servido como referencia inicial.

Planteamiento del problema

Necesidad

Las personas que preparan alimentos durante la semana suelen trabajar con recetas independientes y decisiones tomadas día a día. Este enfoque puede producir duplicidad de compras, olvidos de ingredientes, dificultad para calcular cantidades y poca visibilidad sobre lo que ya existe en la despensa.

Manejo actual

El manejo manual normalmente consiste en consultar recetas, escribir ingredientes en notas o aplicaciones separadas, revisar la despensa y construir una lista de mercado. Cuando existen varias recetas para diferentes días y comidas, el proceso exige consolidar manualmente ingredientes repetidos y ajustar las cantidades disponibles. Esta fragmentación aumenta el esfuerzo y dificulta mantener una relación consistente entre el menú y las compras.

Solución propuesta: Planificador de Menús y Lista de Mercado

La solución propone transformar el flujo de trabajo de la receta suelta al plan de la semana: armar el menú, calcular lo que hace falta y salir con la lista de mercado lista. El usuario podrá buscar recetas por día y comida mediante un calendario semanal; el sistema va a consolidar los ingredientes de todas las recetas seleccionadas, tendrá en cuenta las cantidades disponibles en la despensa y generará la relación de compras pendiente, organizada por categorías.

El propósito no es reemplazar la decisión del usuario sobre qué comer, sino reducir el trabajo repetitivo de organización y cálculo. El sistema funcionará como una herramienta de planificación y apoyo.

Requisitos del sistema

Requisitos funcionales

ID	Requisito	Descripción
RF-01	Crear un plan de menú semanal.	El usuario podrá crear un plan indicando la semana correspondiente.
RF-02	Buscar recetas por día y comida.	Cada receta podrá asignarse a un día de la semana y a un tipo de comida.
RF-03	Reordenar recetas.	El frontend permitirá arrastrar y soltar recetas dentro del calendario.
RF-04	Gestionar recetas.	El sistema permitirá crear, consultar, actualizar y eliminar recetas según los permisos definidos.
RF-05	Gestionar ingredientes.	Cada ingrediente tendrá nombre, unidad base y categoría.
RF-06	Consolidar ingredientes.	El backend sumará las cantidades requeridas por las recetas incluidas en el plan.
RF-07	Descontar despensa.	El cálculo considerará las cantidades disponibles para determinar el faltante.
RF-08	Generar lista de mercado.	El sistema presentará los ingredientes faltantes agrupados por categoría.
RF-09	Marcar compras.	La interfaz permitirá marcar los elementos adquiridos y visualizar su estado.
RF-10	Consultar el plan.	El usuario podrá revisar el menú semanal y la lista calculada antes de realizar la compra.

Tabla 1. Requisitos funcionales principales

Requisitos no funcionales

- RNF-01 Usabilidad: la interfaz deberá presentar el calendario y la lista de mercado de forma clara y consistente.
- RNF-02 Rendimiento: las operaciones habituales de consulta y actualización deberán responder en tiempos adecuados para un proyecto web académico.
- RNF-03 Mantenibilidad: frontend, backend y persistencia deberán mantenerse separados por responsabilidades.
- RNF-04 Seguridad: las credenciales, secretos, claves y contraseñas no deberán almacenarse directamente en el código fuente.
- RNF-05 Integridad: las cantidades de ingredientes y relaciones entre recetas deberán validarse antes de ser persistidas.
- RNF-06 Trazabilidad: los cambios relevantes del proyecto deberán gestionarse mediante control de versiones.
- RNF-07 Portabilidad: el sistema deberá poder ejecutarse en un entorno de desarrollo reproducible y, cuando sea posible, mediante contenedores.

Arquitectura y relación Frontend–Backend

La arquitectura propuesta sigue una separación cliente-servidor. React será responsable de la presentación, interacción del calendario, formularios y visualización de la lista. Spring Boot concentrará las reglas de negocio y expondrá servicios REST. La base de datos relacional almacenará planes, recetas, ingredientes y sus relaciones.

Flujo principal: usuario → React → API REST de Spring Boot → capa de servicios → repositorios → base de datos. Para generar la lista de mercado, el backend obtiene las recetas del plan, consolida sus ingredientes, consulta las existencias registradas en la despensa y calcula la cantidad faltante. El resultado se devuelve al frontend para agrupar y permitir marcar las compras.

Despliegue

Para el entregable académico se propone un despliegue compuesto por frontend, backend y base de datos. El frontend puede ser servido como archivos estáticos mediante un servidor web; Spring Boot ejecutará la API; y la base de datos estará aislada como servicio independiente. El despliegue podrá realizarse localmente o mediante contenedores Docker cuando el equipo lo requiera.

El sistema de la célula 5 integra tres capas fundamentales dentro de la arquitectura de desarrollo: Front-End en React, Back-End en Spring Boot, y una capa de persistencia basada en bases de datos relacionales. La funcionalidad central se divide en tres componentes clave descritos en la especificación del proyecto:

- **Front-End (React):** Calendario semanal interactivo con funcionalidad de arrastrar y soltar (drag and drop) de recetas por día y comida, acompañado de una lista de mercado agrupada por categoría con opción de marcar artículos comprados.

- **Back-End (Spring Boot):** Gestión y creación de planes de menú, consolidación de ingredientes provenientes de múltiples recetas, sustracción automatizada de los artículos disponibles en la despensa y generación final de la lista de compras.
- **Base de Datos:** Modelado relacional estructurado exactamente en cinco tablas principales en español que sostienen las operaciones lógicas de planificación, almacenamiento de recetas, inventario de despensa y consolidación de listas.

Modelo de base de datos

Para mantener el alcance en cinco tablas, el MVP utilizará una estructura relacional compacta. La lista de mercado se calculará dinámicamente a partir del plan, las recetas, sus ingredientes y las existencias registradas. El estado de compra puede mantenerse inicialmente en la capa de presentación o incorporarse posteriormente como persistencia si el alcance del proyecto se amplía.

Tabla	Campos principales	Propósito
plan_menu	id_plan PK; semana_inicio; nombre; estado	Representa una planificación semanal.
menu_item	id_item PK; id_plan FK; id_receta FK; dia_semana; tipo_comida; posicion	Relaciona una receta con un día y una comida dentro del plan.
receta	id_receta PK; nombre; porciones; instrucciones	Catálogo de recetas.
ingrediente	id_ingrediente PK; nombre; unidad_base; categoria; cantidad despensa	Catálogo de ingredientes y existencia disponible.
receta_ingrediente	id_receta_ingrediente PK; id_receta FK; id ingrediente FK; cantidad	Define qué ingredientes requiere cada receta y en qué cantidad.

Tabla 2. Modelo de las cinco tablas del MVP

```

-- =====
-- SAZÓN
-- Célula 5 — Planificador de Menús y Lista de Mercado
-- Database schema — MVP
-- PostgreSQL
-- =====

BEGIN;

-- =====

-- 1. PLAN_MENU
-- =====

CREATE TABLE IF NOT EXISTS plan_menu (
    id_plan    BIGSERIAL PRIMARY KEY,
    semana_inicio DATE NOT NULL,
    nombre     VARCHAR(150) NOT NULL,
    estado     VARCHAR(30) NOT NULL DEFAULT 'ACTIVO',

    CONSTRAINT chk_plan_menu_estado
        CHECK (estado IN ('ACTIVO', 'BORRADOR', 'FINALIZADO', 'ARCHIVADO')),

    CONSTRAINT uq_plan_menu_semana
        UNIQUE (semana_inicio)

```

);

-- =====

-- 2. RECETA

-- =====

CREATE TABLE IF NOT EXISTS receta (

id_receta BIGSERIAL PRIMARY KEY,

nombre VARCHAR(150) NOT NULL,

porciones INTEGER NOT NULL DEFAULT 1,

instrucciones TEXT NOT NULL,

CONSTRAINT chk_receta_porciones

CHECK (porciones > 0)

);

-- =====

-- 3. INGREDIENTE

-- =====

CREATE TABLE IF NOT EXISTS ingrediente (

id_ingrediente BIGSERIAL PRIMARY KEY,

nombre VARCHAR(150) NOT NULL,

```

unidad_base    VARCHAR(30) NOT NULL,

categoria      VARCHAR(80) NOT NULL,

cantidad_despensa  NUMERIC(12,3) NOT NULL DEFAULT 0,


CONSTRAINT chk_ingrediente_despensa

    CHECK (cantidad_despensa >= 0),


CONSTRAINT uq_ingrediente_nombre

    UNIQUE (nombre)

);


-- =====

-- 4. MENU_ITEM

-- =====

CREATE TABLE IF NOT EXISTS menu_item (

    id_item    BIGSERIAL PRIMARY KEY,

    id_plan    BIGINT NOT NULL,

    id_receta   BIGINT NOT NULL,

    dia_semana  SMALLINT NOT NULL,

    tipo_comida VARCHAR(30) NOT NULL,

    posicion   INTEGER NOT NULL DEFAULT 0,

```

```
CONSTRAINT fk_menu_item_plan  
  
FOREIGN KEY (id_plan)  
  
REFERENCES plan_menu (id_plan)  
  
ON DELETE CASCADE,
```

```
CONSTRAINT fk_menu_item_receta  
  
FOREIGN KEY (id_receta)  
  
REFERENCES receta (id_receta)  
  
ON DELETE RESTRICT,
```

```
CONSTRAINT chk_menu_item_dia  
  
CHECK (dia_semana BETWEEN 1 AND 7),
```

```
CONSTRAINT chk_menu_item_posicion  
  
CHECK (posicion >= 0),
```

```
CONSTRAINT chk_menu_item_tipo_comida  
  
CHECK (  
  
    tipo_comida IN (  
  
        'DESAYUNO',  
  
        'ALMUERZO',  
  
        'CENA',  
  
        'MERIENDA'
```

```

    )
),

CONSTRAINT uq_menu_item_slot

    UNIQUE (

        id_plan,

        dia_semana,

        tipo_comida,

        posicion

    )

);

```

```

-- =====
-- 5. RECETA_INGREDIENTE
-- =====

CREATE TABLE IF NOT EXISTS receta_ingredient (

    id_receta_ingredient BIGSERIAL PRIMARY KEY,

    id_receta            BIGINT NOT NULL,

    id_ingredient        BIGINT NOT NULL,

    cantidad             NUMERIC(12,3) NOT NULL,

    CONSTRAINT fk_receta_ingredient_receta

```

```

FOREIGN KEY (id_receta)
REFERENCES receta (id_receta)
ON DELETE CASCADE,

CONSTRAINT fk_receta_ingredient_ingredient
FOREIGN KEY (id_ingredient)
REFERENCES ingrediente (id_ingredient)
ON DELETE RESTRICT,

CONSTRAINT chk_receta_ingredient_cantidad
CHECK (cantidad > 0),

CONSTRAINT uq_receta_ingredient
UNIQUE (
    id_receta,
    id_ingredient
)
);

-- =====
-- INDEXES
-- =====

```

```
CREATE INDEX IF NOT EXISTS idx_menu_item_plan
```

```
ON menu_item (id_plan);
```

```
CREATE INDEX IF NOT EXISTS idx_menu_item_receta
```

```
ON menu_item (id_receta);
```

```
CREATE INDEX IF NOT EXISTS idx_menu_item_dia
```

```
ON menu_item (id_plan, dia_semana);
```

```
CREATE INDEX IF NOT EXISTS idx_receta_ingrediente_receta
```

```
ON receta_ingrediente (id_receta);
```

```
CREATE INDEX IF NOT EXISTS idx_receta_ingrediente_ingrediente
```

```
ON receta_ingrediente (id_ingrediente);
```

```
CREATE INDEX IF NOT EXISTS idx_ingrediente_categoria
```

```
ON ingrediente (categoria);
```

```
COMMIT;
```

```
┌──────────────────┐  
│ plan_menu │
```

id_plan PK
semana_inicio
nombre
estado

|

| 1:N



menu_item
id_item PK
id_plan FK
id_receta FK
dia_semana
tipo_comida
posicion

|

| N:1



--

receta
id_receta PK
nombre
porciones
instrucciones

|

| 1:N



receta_ingredient
id_receta_ingredient PK
id_receta FK
id_ingredient FK
cantidad

|

| N:1



ingrediente

id_ingrediente	
nombre	
unidad_base	
categoria	
cantidad_	
despensa	

Ejemplo de consulta SQL para la consolidación:

Esta consulta calcula la cantidad necesaria de cada ingrediente para un plan semanal concreto:

```

SELECT
    i.id_ingrediente,
    i.nombre,
    i.unidad_base,
    i.categoria,
    SUM(ri.cantidad) AS cantidad_requerida,
    i.cantidad_despensa,
    GREATEST(
        0,
        SUM(ri.cantidad) - i.cantidad_despensa
    )

```

```

        ) AS cantidad_a_comprar
FROM menu_item mi
JOIN receta_ingrediente ri
    ON ri.id_receta = mi.id_receta
JOIN ingrediente i
    ON i.id_ingrediente = ri.id_ingrediente
WHERE mi.id_plan = :idPlan
GROUP BY
    i.id_ingrediente,
    i.nombre,
    i.unidad_base,
    i.categoria,
    i.cantidad_despensa
HAVING GREATEST(
    0,
    SUM(ri.cantidad) - i.cantidad_despensa
) > 0
ORDER BY
    i.categoria,
    i.nombre;

```

Regla de cálculo de la lista

Para cada ingrediente, el backend calculará: cantidad requerida total = suma de las cantidades de las recetas del plan. Cantidad a comprar = máximo entre cero y la cantidad requerida total menos la cantidad disponible en despensa. Si el resultado es cero, el ingrediente no deberá aparecer como pendiente de compra.

Diagramación

Casos de uso

Actores: Usuario y Sistema. Casos de uso principales: crear plan semanal, consultar plan, agregar receta al día/comida, mover receta, gestionar recetas, gestionar ingredientes/despensa, consolidar ingredientes, generar lista de mercado y marcar producto como comprado.

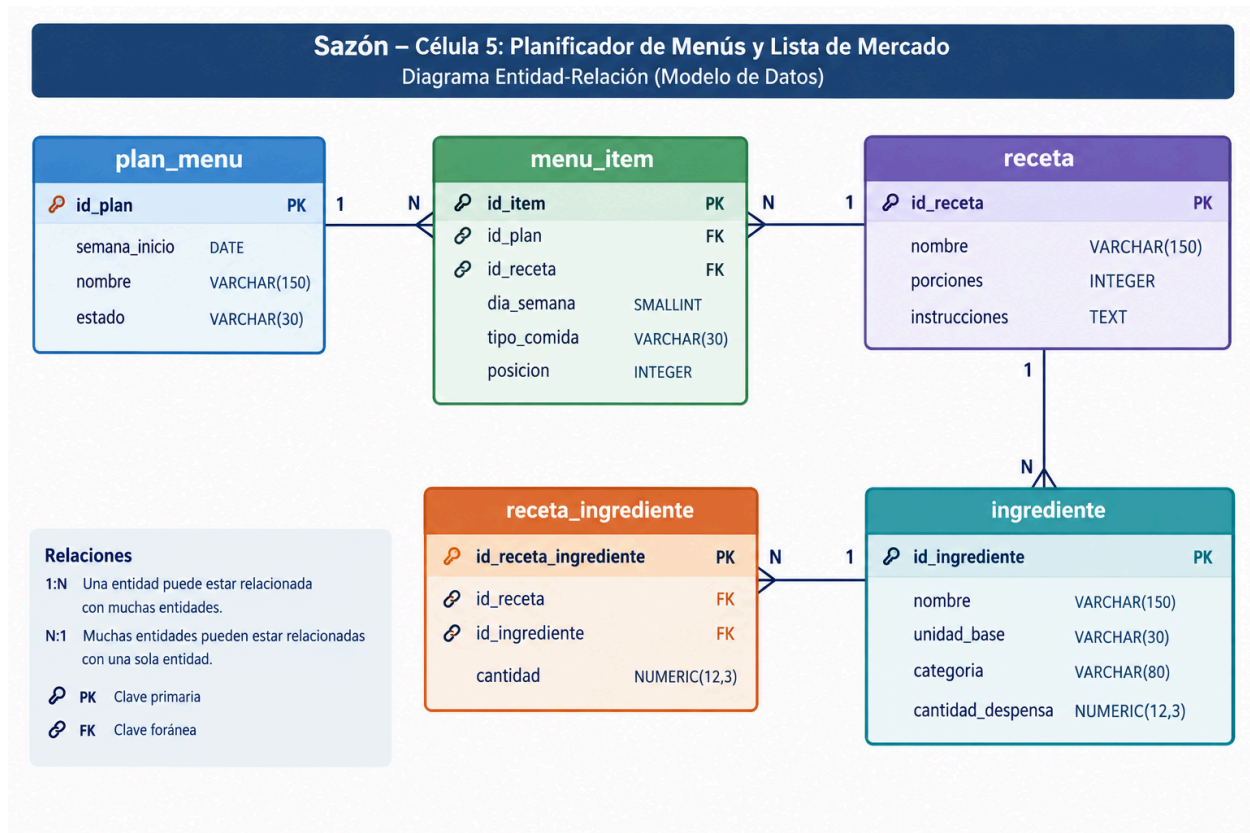
Ilustración 1. Casos de uso principales del sistema



Modelo relacional

El siguiente diagrama esquemático ilustra la arquitectura relacional y el flujo de conexión entre las cinco tablas principales. Se detallan las claves primarias (PK), claves foráneas (FK) y las cardinalidades que permiten el funcionamiento del planificador y la generación automática de la lista de mercado:

Ilustración 2. Modelo relacional de las cinco tablas



Alcance

Entregable final

El entregable final corresponde al proyecto completo desarrollado dentro del marco de la Célula 5. Debe integrar frontend, backend, base de datos, documentación técnica y funcional, pruebas básicas y un mecanismo de ejecución o despliegue reproducible, de acuerdo con las condiciones académicas establecidas.

Inclusiones

- Aplicación web funcional para planificación semanal.
- Calendario con organización de recetas por día y comida.
- Gestión básica de recetas e ingredientes.
- Consolidación de ingredientes de varias recetas.
- Cálculo de faltantes a partir de la despensa.
- Lista de mercado agrupada por categoría.
- Marcación visual de productos comprados.
- API backend desarrollada con Spring Boot.
- Persistencia relacional con cinco tablas para el MVP.
- Documentación, diagramas y evidencia del desarrollo.

Exclusiones

- Integración con supermercados, pasarelas de pago o compras reales.
- Recomendaciones médicas, nutricionales o dietas clínicas.
- Inteligencia artificial para generación automática de dietas, salvo que sea incorporada expresamente mediante una ampliación del alcance.
- Aplicaciones móviles nativas.

- Uso de marcas, logotipos, manuales de identidad o activos propietarios de terceros sin autorización.

- Representación del producto como una solución oficial de Grupo Nutresa o de cualquiera de sus empresas.

Stack tecnológico

Tecnología / herramienta	Uso en el proyecto
Java	Lenguaje principal del backend.
Spring Boot	Framework para API REST, servicios y lógica de negocio.
JavaScript	Lenguaje principal del frontend.
React	Biblioteca para construir la interfaz de usuario.
CSS	Presentación y estilos de la aplicación.
SQL	Consultas y gestión de la base de datos relacional.
PostgreSQL / motor relacional equivalente	Persistencia de la información del MVP, según disponibilidad y decisión del equipo.
Git y GitHub/GitLab	Control de versiones y colaboración.
Docker	Empaquetado y ejecución reproducible, cuando se utilice.
Canva	Herramienta de apoyo para piezas visuales o documentación gráfica no propietaria.
Herramienta UML/ER	Diseño de casos de uso y modelo relacional.

Tabla 3. Stack tecnológico propuesto

Implementación y condiciones legales

Marco Legal y Contractual:

El desarrollo académico y práctico del proyecto se encuentra contextualizado dentro del marco normativo colombiano aplicable al contrato de aprendizaje. Para efectos de esta documentación, se emplean como referencias públicas la Ley 789 de 2002 y la normativa vigente que desarrolla la figura del contrato de aprendizaje.

De acuerdo con la regulación vigente, el contrato de aprendizaje constituye una modalidad especial de vinculación laboral orientada a la formación teórico-práctica y relacionada con el programa de formación del aprendiz.

La presente referencia se incorpora únicamente con fines académicos y de contextualización jurídica. No se incluyen datos personales, condiciones particulares, información confidencial ni elementos identificativos del contrato individual suscrito por el aprendiz.

Naturaleza académica y delimitación de marca

El proyecto se define como un desarrollo académico y formativo de la Célula 5. No constituye una aplicación oficial, patrocinada, licenciada, avalada ni desarrollada para Grupo Nutresa, salvo que exista una autorización escrita y verificable del titular correspondiente. La referencia conceptual que haya podido originar el ejercicio académico no equivale por sí misma a una autorización para utilizar signos distintivos, identidad visual, material corporativo, contenidos internos o información reservada.

Por prevención, la versión final deberá utilizar una identidad visual propia: nombre del producto definido por el equipo, logotipo propio, paleta de colores propia, tipografías y recursos

gráficos con licencia o de creación propia. No se deberán incorporar logotipos, eslóganes, capturas corporativas, fotografías propietarias, manuales de marca, interfaces internas ni elementos que puedan hacer creer que existe una relación empresarial con Grupo Nutresa.

Propiedad industrial y uso de signos distintivos

En Colombia, la Decisión 486 de 2000 reconoce al titular de una marca registrada facultades para impedir determinados usos no consentidos de signos idénticos o semejantes en los supuestos previstos por la norma. La Superintendencia de Industria y Comercio tramita asuntos relacionados con infracciones de propiedad industrial y competencia desleal. Por ello, el proyecto debe evitar un uso que pueda generar confusión sobre el origen empresarial, patrocinio, autorización o vinculación del producto. (Comunidad Andina, 2000; Superintendencia de Industria y Comercio [SIC], s. f.).

Es importante distinguir entre copiar una identidad corporativa y utilizar una combinación cromática. Una paleta de colores, considerada aisladamente, no significa automáticamente que exista una infracción marcaria. El riesgo jurídico depende del conjunto de elementos utilizados, del signo concreto, de los productos o servicios, de la posibilidad de confusión, del contexto y de la conducta. La Ley 256 de 1996 contempla, entre otros, actos de confusión, imitación y explotación de la reputación ajena cuando se configuran sus presupuestos legales.

Posibles consecuencias jurídicas

No es correcto afirmar que una persona, una institución educativa o un grupo de aprendices serán automáticamente sancionados por desarrollar una interfaz académica con colores parecidos a los de una empresa. Para que exista responsabilidad deben analizarse los

hechos concretos, la titularidad de los derechos, el tipo de uso, la autorización existente, el contexto y las normas aplicables.

Sin embargo, un uso no autorizado de signos distintivos que llegue a configurar una infracción puede dar lugar a acciones de propiedad industrial y, según el caso, a medidas judiciales. La SIC informa que conoce procesos de infracción de derechos de propiedad industrial y acciones relacionadas con competencia desleal. En casos de competencia desleal pueden discutirse conductas como confusión, imitación o explotación de la reputación ajena.

Además, el Código Penal colombiano contempla el delito de usurpación de marcas y patentes en el artículo 306, cuya aplicación exige los elementos establecidos por la norma. La disposición actualmente publicada por Función Pública establece, para quien fraudulentamente utilice un nombre comercial, enseña, marca u otros derechos protegidos en los términos allí descritos, pena de prisión de cuatro a ocho años y multa de 26,66 a 1.500 salarios mínimos legales mensuales vigentes. Esto no significa que cualquier proyecto estudiantil o cualquier semejanza visual configure automáticamente ese delito; el elemento de fraude y los demás requisitos legales deben ser demostrados. (Ley 599 de 2000, art. 306).

Derechos de autor sobre el software

El software también se encuentra dentro del ámbito de protección del derecho de autor. La Dirección Nacional de Derecho de Autor señala que el registro del software es declarativo y sirve como medio de prueba, mientras que la protección nace con la creación de la obra. En consecuencia, el código, documentación, diseños originales y otros elementos protegibles deben gestionarse respetando la titularidad y las licencias correspondientes.

Responsabilidad de aprendices e institución

La responsabilidad individual de los aprendices, la responsabilidad institucional y cualquier consecuencia contractual no deben presumirse. Deben revisarse el contrato o acuerdo de aprendizaje, reglamento institucional, políticas de propiedad intelectual, instrucciones del proyecto y cualquier acuerdo de confidencialidad aplicable. Si existiera información empresarial confidencial, su utilización en un proyecto académico podría generar un problema distinto al simple uso de una marca.

Por esta razón, la posición preventiva del documento es que ningún integrante del equipo debe presentar el sistema como un producto de Grupo Nutresa, utilizar activos corporativos no autorizados o incorporar información reservada. Si una actividad académica exige utilizar una identidad corporativa concreta, se recomienda solicitar a la institución una instrucción formal y, cuando corresponda, autorización del titular de los derechos antes de publicar o desplegar el material.

Cláusula de delimitación propuesta

“Este proyecto es un desarrollo académico correspondiente a la Célula 5. No representa ni pretende representar una solución oficial, producto, servicio, filial, proveedor, patrocinador o iniciativa de Grupo Nutresa. Las marcas, nombres comerciales, logotipos, identidad visual, contenidos y demás activos de terceros pertenecen a sus respectivos titulares. La solución utiliza una identidad propia y no incorpora activos de terceros salvo aquellos utilizados bajo autorización o licencia válida.”

Organización del equipo y metodología

El proyecto está planteado para un equipo de cinco personas. Para efectos de gestión se utilizará Scrum como marco de trabajo, con planificación de tareas, priorización del backlog, seguimiento de avances, revisión del incremento y retrospectiva.

En la situación actual, la mayor parte de la ejecución está siendo asumida por un solo integrante, quien está desempeñando simultáneamente funciones de Scrum Master y desarrollo. Esta circunstancia debe registrarse como una condición real del proyecto, no como una asignación ideal de roles. La documentación debe distinguir entre la composición nominal del equipo y la participación efectiva observada durante cada periodo de trabajo.

Para proteger la trazabilidad y la responsabilidad profesional, las tareas deberán registrarse con fecha, responsable, evidencia y estado. Esto permite demostrar qué actividades fueron realizadas, cuáles quedaron pendientes y qué decisiones fueron tomadas durante el desarrollo.

Conclusiones como equipo de desarrollo

El análisis realizado permite concluir que el problema puede abordarse con una arquitectura web relativamente sencilla, separando la presentación, la lógica de negocio y la persistencia. El núcleo funcional se concentra en tres operaciones: planificar la semana, consolidar ingredientes y calcular el faltante para producir una lista de mercado útil.

El modelo de cinco tablas permite mantener un MVP coherente sin introducir complejidad innecesaria. Las reglas de cálculo pueden mantenerse en el backend, evitando que el frontend sea responsable de decisiones de negocio.

La delimitación del alcance es fundamental para evitar que el proyecto se convierta en una plataforma de compras o nutrición de mayor complejidad. También es fundamental mantener una identidad visual independiente de terceros y documentar expresamente el carácter académico del desarrollo.

Reflexión final sobre el trabajo en equipo

El desarrollo de un proyecto de software en equipo requiere comunicación, distribución de responsabilidades, acuerdos técnicos y seguimiento. Cuando estos elementos no funcionan de manera coordinada, una parte desproporcionada de la carga puede concentrarse en uno de los integrantes.

En el contexto actual, aunque el equipo está compuesto por cinco personas, la ejecución efectiva está siendo asumida principalmente por un integrante. Esta situación representa un reto tanto técnico como de gestión: además de programar, es necesario organizar prioridades, mantener la documentación y procurar que el producto llegue a un estado demostrable. El proyecto corresponde al **Grupo 5 — Planificador de Menús y Lista de Mercado**. Las responsabilidades se organizan de acuerdo con los roles establecidos en la metodología Scrum y con la estructura definida en los materiales académicos utilizados como referencia.

Durante el desarrollo se identificaron situaciones en las que algunas responsabilidades no quedaron claramente asignadas o comunicadas. Asimismo, se registraron solicitudes de revisión y seguimiento cuya respuesta no siempre permitió determinar una tarea concreta, responsable, criterio de aceptación o fecha de cumplimiento.

Por esta razón, el presente cuadro tiene como finalidad establecer una referencia documental de los integrantes y sus roles, facilitando posteriormente la trazabilidad de las contribuciones mediante evidencias como commits, documentos, mensajes del grupo, revisiones y entregables.

Nota: La asignación de roles y responsabilidades debe contrastarse con el documento académico o material de referencia utilizado por el curso. Las

actividades específicas de cada integrante deben determinarse mediante evidencia verificable del trabajo realizado y no únicamente mediante la denominación del rol.

La experiencia también permite reconocer que Scrum no consiste únicamente en utilizar reuniones o nombres de roles. Su valor está en hacer visible el trabajo, identificar impedimentos y mantener un incremento verificable. La reflexión final es que un proyecto puede avanzar incluso en condiciones de participación desigual, pero la sostenibilidad del proceso exige transparencia sobre las contribuciones y una adecuada gestión de responsabilidades.

Referencias:

El desarrollo de este proyecto contó con el apoyo de los materiales de instrucción, documentos, presentaciones, guías, ejemplos y orientaciones metodológicas proporcionados por el docente del curso y la institución educativa.

Estos materiales fueron utilizados como referencias académicas para la elaboración de la documentación de requisitos, la metodología Scrum, las historias de usuario, el diseño del sistema, las actividades de desarrollo de software y la documentación general del proyecto.

Nota: La información bibliográfica específica de los materiales proporcionados por el docente deberá completarse utilizando el título, autor, institución, año y/o URL que aparezcan en los materiales originales.

La documentación del proyecto también reconoce el contexto contractual y académico aplicable mediante el documento:

Contrato de Aprendizaje Laboral Especial a Término Fijo.

Este documento se incluye como referencia legal y contractual para el contexto en el cual se desarrollan las actividades de aprendizaje y prácticas asociadas al proyecto.

El documento contractual deberá ser citado de acuerdo con la versión oficial proporcionada por la institución, empresa o entidad contratante. Su fecha exacta, entidad emisora, versión y demás datos de identificación deberán conservarse conforme aparecen en el documento oficial, evitando reconstruir dicha información de memoria.

Las referencias incluidas en este documento tienen como finalidad proporcionar trazabilidad respecto de las fuentes utilizadas durante el desarrollo académico del proyecto.

Los materiales proporcionados por el docente, la documentación institucional, los documentos legales o contractuales y la documentación técnica deberán distinguirse de acuerdo con su respectiva naturaleza.

Ninguna fuente deberá presentarse como autoridad académica o legal sin que pueda verificarse su documento original y su información bibliográfica correspondiente.

Comunidad Andina. (2000). Decisión 486: Régimen común sobre propiedad industrial.

<https://www.comunidadandina.org/DocOficialesFiles/Gacetas/Gace600.pdf>

Dirección Nacional de Derecho de Autor. (s. f.). Software.

<https://www.derechodeautor.gov.co/es/otros/software>

Dirección Nacional de Derecho de Autor. (s. f.). Registro de software.

<https://www.derechodeautor.gov.co/es/atencion-y-servicios-a-la-ciudadania/registro-de-o-bras/registro-de-software/registro-de-software>

Función Pública. (2000). Ley 599 de 2000: Por la cual se expide el Código Penal.

<https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?0=&i=6388>

Función Pública. (1982). Ley 23 de 1982: Sobre derechos de autor.

<https://www1.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=3431>

Superintendencia de Industria y Comercio. (s. f.). Competencia desleal y propiedad industrial.

<https://sedeelectronica.sic.gov.co/temas/asuntos-jurisdiccionales/proteccion-del-consumidor/competencia-desleal-y-propiedad-industrial>

ZAPATA, Jaime. *Backlog del Producto*. CESDE, material pédagogique du cours, [s. d.].

Disponible sur :

<https://docs.google.com/spreadsheets/d/16S6kVMb1RMUb8icqdKf1Ex3PK7UFzyMs/edit?gid=651647471#gid=651647471>

ZAPATA, Jaime. *Asignación – Metodologías*. CESDE, material pédagogique du cours, [s. d.].

Disponible sur :

https://docs.google.com/spreadsheets/d/1I85dGy3mablUgxN85FjZApJDjE7_gN2L/edit?gid=1472682486#gid=1472682486

COLOMBIE. MINISTERIO DEL TRABAJO. *Decreto 223 de 2026: por el cual se modifican y adicionan disposiciones relacionadas con la vinculación formativa y el contrato de aprendizaje*.

Bogotá D. C.: Ministerio del Trabajo, 2026. Disponible sur :

<https://www1.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=272856>

COLOMBIE. CONGRESO DE LA REPÚBLICA. *Ley 789 de 2002: por la cual se dictan normas para apoyar el empleo y ampliar la protección social y se modifican algunos artículos del Código Sustantivo de Trabajo*. Bogotá D. C., 2002. Disponible sur :

https://www.funcionpublica.gov.co/eva/gestornormativo/norma_pdf.php?i=6778

SERVICIO NACIONAL DE APRENDIZAJE — SENA. *Compilación normativa: contrato de aprendizaje*. Bogotá D. C.: SENA. Disponible sur :

https://normograma.sena.edu.co/compilacion/docs/decreto_0223_2026.htm

Anexo A. Matriz resumida de alcance

Producto: Planificador de Menús y Lista de Mercado. Célula: 5. Tipo: proyecto académico. Estado documental: base para levantamiento y desarrollo de requisitos.

Anexo B. Nota de revisión legal

Este apartado tiene finalidad preventiva y académica. No sustituye el análisis de un abogado sobre contratos, derechos de propiedad industrial, derechos de autor, confidencialidad o responsabilidad institucional. Antes de una publicación externa, uso comercial, presentación pública con marcas de terceros o despliegue para una empresa, deberán verificarse las autorizaciones y condiciones contractuales aplicables.